

RISC-V memcpy / memset 算法优化报告

实现文件: submission/src/memcpy-vector.S, submission/src/memset-vector.S

目标环境: RV64GC + RVV 1.0 + Zicboz, 默认 Zicboz cache block 为 64 字节

1. 设计出发点

这两个函数属于标准库里最热的基础路径。优化时不能只盯住大块带宽, 还要照顾大量随机长度调用。因此实现没有采用单一路径, 而是把输入长度拆成小块、中等长度和大块三类。小块尽量避免启动 RVV, 中等长度交给 vsetvli 自适应处理尾部, 大块再使用多组向量寄存器展开来提高吞吐。memset 另外单独处理 value 为 0 的情况, 因为 Zicboz 的 cbo.zero 在清零场景下比普通 store 更合适。

场景	选择的路径	原因
几十到一百多字节的短调用	整数寄存器 load/store 或 store 展开	避免 vsetvli 和向量状态启动成本, 对随机长度更稳定。
中等长度	单组 RVV 循环	用 vsetvli 的返回值作为本轮处理长度, 不需要额外尾部代码。
大块连续区域	v0/v8/v16/v24 多组寄存器展开	减少循环分支占比, 增加每轮 load/store 字节数。
memset 清零	64 字节对齐后使用 cbo.zero	一条指令清理一个 cache block, 对 zero benchmark 更直接。

2. memcpy-vector.S 的主逻辑

memcpy 的实现保持前向复制。入口首先检查长度是否为 0, 直接返回, 这样不会让空复制进入后面的标量路径。a0 始终保存原始目的地址, 后续用 a4/a5 作为移动指针, 保证返回值符合 memcpy 语义。

2.1 小尺寸 first/last 复制

短复制的核心思想是只复制区间的开头和结尾。比如 8-15 字节时, 先从 src 读 8 字节写到 dst, 再从 src+n-8 读 8 字节写到 dst+n-8。两次写入可能重叠, 但重叠发生在目标区间内部, 不会越界。2-3 字节用 lhu/sh, 4-7 字节用 lw/sw, 8 字节以上用 ld/sd。这样比逐字节循环少很多分支, 也没有 RVV 启动开销。

在 65-192 字节区间, 完全使用 first/last 会造成 65 字节附近重复搬运过多数据, 所以这里改成 8-byte 标量循环。循环复制完整的 8 字节块, 如果最后还有余数, 再用最后一个 ld/sd 覆盖尾部。这一段主要服务随机长度调用, 因为随机测试里大量长度落在向量和短复制之间, 此时启动 RVV 未必划算。

```
memcpy length dispatch:
n == 0 -> return
1..64 -> scalar first/last
65..192 -> scalar 8-byte loop, then last ld/sd
193..(4*VLMAX-1) -> simple RVV loop
>= 4*VLMAX -> four-register RVV bulk loop
```

2.2 中等长度 RVV 循环

进入向量路径后, 代码使用 e8,m8 作为基本配置。每轮执行 vsetvli ivl,num,e8,m8,ta,ma, 硬件返回本轮实际可处理的字节数 ivl。随后执行一次 vle8.v 和一次 vse8.v, 再将 src、dst、num 分别按 ivl 更新。只要 num 不为 0 就继续循环。这个结构的好处是尾部自然由 vsetvli 处理, 不需要额外写 byte tail。

2.3 大块四组寄存器展开

大块路径先用 vsetvli 获取 VLMAX, 然后把每轮处理量设为 4*VLMAX。循环中依次用 v0、v8、v16、v24 加载四段连续数据, 再按相同顺序存回目的地址。由于 LMUL=m8, 这四组寄存器正好覆盖互不冲突的向量寄存器组。相比单组循环, 四组展开减少了分支和计数指令在总周期中的比例,

更适合大块 memcpy 的带宽路径。

3. memset-vector.S 的主逻辑

memset 的入口先保存 dst 到 dst_ptr, 然后处理两个关键分支: 长度为 0 直接返回; value 先与 255 做 andi, 只保留低 8 位。这个截断必须放在前面, 否则传入大于 255 或负数的 int 时会破坏 memset 的标准语义。如果截断后的 value 为 0, 直接进入 zero_path; 否则走非零填充路径。

3.1 非零 memset

非零小块采用标量写。和 memcpy 类似, 1-64 字节使用 first/last 写法, 65-192 字节使用 8-byte store 循环。只有在进入标量路径时, 才把低 8 位 value 扩展成 64 位重复字节 pattern。这样做是为了避免大块 RVV 路径无意义地执行移位和或操作, 因为 e8 向量 store 最终只需要每个元素的低 8 位。

中等非零长度进入 vector_fill。代码用 vmv.v.x 将 value 广播到 v0, 然后每轮用 vsetvli 得到 ivl 并执行 vse8.v。大块非零路径则提前广播到 v0/v8/v16/v24 四组向量寄存器, 每轮连续执行四次向量 store, 每次推进 VLMAX 字节。

3.2 memset zero 与 Zicboz

value 为 0 时单独处理。目标平台按 64 字节 cache block 设计, 因此代码中固定使用 CBO_BLOCK_SIZE=64、CBO_BLOCK_MASK=63、CBO_BLOCK_SHIFT=6。小于 64 字节的清零走标量尾部; 达到 64 字节后, 先把 dst_ptr 推进到 64 字节对齐位置, 再执行 cbo.zero。

未对齐前缀没有使用 RVV, 而是直接用 8 个 sd zero 清理从 dst_ptr 开始的 64 字节窗口。随后按实际对齐距离扣减 num 并推进指针。这会让部分字节和之后的 cbo.zero 发生重叠, 但写入仍位于目标区间内。这样换来的是清零路径不必为前缀启动向量状态。cbo.zero 主循环按 4 个 block 展开, 每轮清 256 字节, 剩余不足 4 个 block 时再进入单 block 循环。最后 1-63 字节仍用标量清尾。

```
memset dispatch:
n == 0 -> return
(value & 255) == 0, n < 64 -> scalar zero tail
(value & 255) == 0, n >= 64 -> align, cbo.zero loop, scalar tail
nonzero n <= 192 -> scalar pattern stores
nonzero medium -> vmv.v.x + vsetvli loop
nonzero large -> v0/v8/v16/v24 store unroll
```

4. 关键取舍

取舍点	采用方案	说明
标量/RVV 阈值	192 字节	偏向随机长度测试。若某个平台 RVV 启动极低, 这个值可以降低到 128。
小块非对齐访问	使用 ld/sd/lw/sw	源码声明 unaligned_access,1, 假设目标处理器非对齐访问较快。
RVV 元素宽度	e8,m8	内存函数按字节处理, e8 避免额外组合; m8 尽量扩大单次 VLMAX。
zero block size	固定 64 字节	比赛目标平台按 64 字节 Zicboz block 使用, 不加入运行时探测开销。
大块展开	四组向量寄存器	在寄存器占用和循环展开之间取平衡, 避免过度展开导致代码膨胀。

5. 正确性说明

两个函数都保持 a0 为原始目的地址, 因而返回值正确。memcpy 按标准语义不处理重叠内存, 实现采用前向复制。first/last 路径虽然可能在目标区间内部重叠写入, 但不会访问区间外地址。65-192 字节循环的最后一次 ld/sd 使用 end-8, 同样保证访问落在区间内。memset 在入口截断 value 的低 8 位, zero 和非零路径都使用同一语义来源。

RVV 路径的尾部全部交给 vsetvli。每轮根据返回的 ivl 更新指针和剩余长度, 因此不会出现手写尾部循环常见的边界问题。Zicboz 路径只在 dst_ptr 对齐到 64 字节后执行 cbo.zero; 未对齐前缀和最后尾部都用标量 store 处理。

6. 小结

整体实现可以概括为: 短长度用整数寄存器减少启动成本, 中等长度用 vsetvli 保持代码简单且尾部安全, 大块长度用四组 RVV 寄存器拉高吞吐, memset zero 再额外使用 Zicboz 做 cache block 级清零。这套分支结构不是为了追求形式上的复杂, 而是让不同长度区间分别走各自更合适的执行单元。